

Sensory-motor control with large language models via iterative policy refinement

Received: 6 June 2025

Accepted: 24 February 2026

Published online: 15 March 2026

Cite this article as: Carvalho J.T. & Nolfi S. Sensory-motor control with large language models via iterative policy refinement. *Sci Rep* (2026). <https://doi.org/10.1038/s41598-026-42091-0>

Jonata Tyska Carvalho & Stefano Nolfi

We are providing an unedited version of this manuscript to give early access to its findings. Before final publication, the manuscript will undergo further editing. Please note there may be errors present which affect the content, and all legal disclaimers apply.

If this paper is publishing under a Transparent Peer Review model then Peer Review reports will publish with the final article.

ARTICLE IN PRESS

Sensory-Motor Control with Large Language Models via Iterative Policy Refinement

Jonata Tyska Carvalho^{1,2,*} and Stefano Nolfi²

¹Federal University of Santa Catarina (UFSC), Informatics and Statistics Department, Florianópolis, Brazil

²Institute of Cognitive Sciences and Technologies (ISTC-CNR), Rome, Italy

*jonata.tyska@ufsc.br

ABSTRACT

We propose a method that enables large language models (LLMs) to control embodied agents through the generation of control policies that directly map continuous observation vectors to continuous action vectors. At the outset, the LLMs generate a control strategy based on a textual description of the agent, its environment, and the intended goal. This strategy is then iteratively refined through a learning process in which the LLMs are repeatedly prompted to improve the current strategy, using performance feedback and sensory-motor data collected during its evaluation. The method is validated on classic control tasks from the Gymnasium library and the inverted pendulum task from the MuJoCo library. The approach proves effective with relatively compact models such as GPT-oss:120b and Qwen2.5:72b. In most cases, it successfully identifies optimal or near-optimal solutions by integrating symbolic knowledge derived through reasoning with sub-symbolic sensory-motor data gathered as the agent interacts with its environment.

1 Introduction

Large Language Models (LLMs) have emerged as powerful tools for developing embodied agents capable of exhibiting goal-directed behaviors through interaction with external environments. This capability stems from their ability to acquire extensive knowledge and complex reasoning skills via observational learning, wherein they process and internalize patterns from large-scale textual and multimodal data. The knowledge and skills obtained in this manner extend beyond purely linguistic tasks and can be effectively repurposed to control embodied agents. This versatility positions LLMs as a promising foundation for training and deploying autonomous systems across a wide range of domains, including robotics, virtual assistants, and interactive simulations.

The straightforward approach to leveraging LLMs for embodied control involves equipping agents with a repertoire of motor primitives—basic motion control modules—that enable the execution of elementary actions required to achieve basic objectives, such as navigating to a target or grasping an object. In this approach, the LLM is prompted to generate descriptions with the sequence of necessary basic actions that permit the achievement of the goal. These descriptions are then translated into concrete behaviors by activating the corresponding sequence of motor primitives. For example, Huang et al. (2022)¹ showed that GPT-3² and Codex³ could produce plausible-albeit sometimes impractical-action plans when provided with a task description and a single example of a suitable action plan.

Simple frameworks of this kind—relying solely on language-based instructions without incorporating environmental perception or the ability to observe change during action executions—can be extended and enhanced through the use of multimodal LLMs operating dynamically. These models are capable of processing visual data, either encoded as specialized embedding vectors⁴ or represented as textual descriptions generated by vision-language models⁵. Furthermore, the framework can be improved by enabling the model to monitor the environment during action execution and to adjust its action plan as needed dynamically⁶.

However, frameworks based on predefined motor primitives face intrinsic limitations. The behavior of embodied agents often exhibits quasi-modular organization^{7,8}, meaning that overall behavior can be decomposed into sub-behaviors or components. However, these components lack sharp boundaries and must be finely tuned to the characteristics of preceding and subsequent actions. Consequently, designing or training primitive behaviors independently can hinder behavioral integration and compromise overall performance⁸. Such approaches are, therefore, only suitable for domains characterized by low dynamical complexity and limited interdependence among sequential actions.

In this article, we explore an alternative approach in which LLMs generate control strategies that produce continuous action vectors directly, encoding the temporal evolution of the agent’s actuator states and eliminating the need for predefined motor primitives. Furthermore, we investigate how such models can improve their ability to produce goal-directed behaviors through

a learning process realized through iterated prompting.

The remainder of the paper is organized as follows: Section 2 reviews related work; Section 3 describes our proposed method; Section 4 presents our experimental results; and Section 5 provides concluding remarks. A comprehensive account of the experimental setup, the obtained results, and the software used for replication and further experimentation is included in the appendices provided in the supplementary material.

2 Related work

The most direct and widely adopted framework for sensory–motor learning is Reinforcement Learning (RL). In RL, an agent acquires a control policy by interacting with its environment and optimizing expected cumulative reward through evaluative feedback⁹. This paradigm has become a standard approach for learning continuous control policies in both simulated and real-world domains, including robotics, manipulation, and locomotion^{10–13}. RL provides a natural baseline for studying sensory–motor skill acquisition, as it directly addresses the problem of mapping sensory observations to motor actions through experience-driven optimization.

Reinforcement learning encompasses a broad family of methods that differ along several dimensions. These include online versus offline (batch) learning, where policies are either optimized through ongoing interaction or learned solely from previously collected datasets^{14–16}, as well as model-free and model-based approaches, which respectively learn policies directly from experience or leverage an explicit model of the environment’s dynamics. Despite their differences, all RL variants ultimately rely on interaction data generated by embodied agents operating in their target environments, either directly during training or indirectly through datasets collected by some behavior policy. While this interaction-centric nature ensures task- and morphology-specific learning, it can also result in high interaction data requirements, particularly for complex control problems.

In this work, we investigate LLM-based systems can support alternative forms of sensory–motor learning. Unlike standard RL, LLMs possess extensive prior knowledge and reasoning capabilities acquired through large-scale pretraining, which can be exploited to initialize the agent’s control policy and to propose structured variations that are likely to yield performance improvements. This could potentially accelerate the learning process and reduce the number of required interactions with the environment.

As stated in the introduction, this article focuses on methods that utilize LLMs to generate control policies that directly govern the state of the agent’s actuators, without relying on predefined motor primitives. Within this category, different approaches have been proposed that vary in terms of training requirements, the use of demonstration data, and the integration of iterative learning mechanisms. An initial method of this kind was developed by Brohan et al. (2023)¹⁷, who fine-tuned vision-language models (VLMs) to generate specialized action tokens that encode the state of an agent’s actuators over time. The authors demonstrated that this method enables a robotic arm equipped with a camera to respond appropriately to task requests such as "Pick up the bag about to fall off the table" when the agent observes its own arm and several bags positioned on a table, or "Put the strawberry into the correct bowl" when it observes its arm, a strawberry, a bowl of bananas, and a bowl of strawberries.

More specifically, Brohan et al. trained the PaLI-X¹⁸ and PaLM-E⁴ models to respond to prompts formatted as: "What action should the robot take to [task instruction]?" paired with task-specific visual inputs. The models were trained to produce vectors of special action tokens that encode the gripper’s extension and the angular positions of the robotic arm’s six joints over time, ensuring the execution of the desired behavior. These joint positions were discretized into 256 bins, each represented by a distinct action token.

Training data were collected from 13 robots operating in a kitchen-office environment over a 17-month period¹⁷. Each robot demonstration trajectory was annotated with a natural language description of the corresponding task. Beyond replicating the demonstrated behaviors, the trained models exhibited strong generalization capabilities, successfully performing novel tasks not encountered during training. However, a key limitation of this approach lies in its dependence on large-scale, labor-intensive demonstration datasets.

In contrast, other researchers have explored prompting large language models (LLMs) to generate action vectors directly without using fine-tuning. Wang et al. (2024)⁶ employed GPT-4¹⁹ to control a simulated quadruped robot with 12 degrees of freedom (DOF), enabling it to walk on a flat surface. The model received continuous observation vectors as input and produced corresponding action vectors at a frequency of 200 Hz. This was achieved by prompting GPT-4 with (i) a description of the robot and task, (ii) an explanation of the observation and action vectors format, and (iii) examples of correct observation and action vector pairs. This prompt induced in-context learning, enabling the model to generate effective walking actions in response to new observations.

Notably, LLMs prompted in this manner can synthesize control strategies by integrating qualitatively distinct forms of information: (i) symbolic information conveyed through textual descriptions of the robot, environment, and task objectives, (ii) symbolic information produced via reasoning, and (iii) sub-symbolic information contained in continuous sensory-motor data.

Furthermore, Mirchandani et al. (2023)²⁰ showed that LLMs can generate appropriate actions through in-context learning using only sensory-motor data, without receiving any explicit information about the robot, the environment, or the task objective. In their experiments, an LLM successfully controlled a Panda robotic arm²¹ in a “Marker in a Cup” task, relying solely on demonstration data. As in the previous work mentioned above, in-context learning enabled the model to produce appropriate action vectors in response to new observation vectors.

Moreover, Mirchandani et al. (2023)²⁰ demonstrated that LLMs can leverage their in-context learning capabilities to improve their control strategies over multiple learning cycles. This was achieved by repeatedly prompting the model across several iterations and updating the demonstration data in each cycle. The updated dataset was composed of the best behavioral samples selected from both human demonstrations and samples generated by the models in previous iterations.

This iterative learning process can be further enhanced by incorporating reward signals into the prompt—scalar values that quantify how well each demonstrated or generated behavior achieves the intended task objective.

In this article, we propose an LLM-based methodology for generating and iteratively refining control strategies that directly govern the state of an agent’s actuators. Our approach enables models to learn autonomously without relying on human-provided demonstrations. Instead, the model learns from a combination of (i) linguistic descriptions of the robot, its environment, and the task objectives and (ii) sensory-motor data and rewards generated throughout the learning process.

Our approach integrates three key ideas: (i) leveraging LLMs’ reasoning and in-context learning capabilities to derive control strategies from textual descriptions, as demonstrated by Wang et al. (2023)⁶; (ii) complementing this with learning from sensory-motor samples, as shown by²⁰; and (iii) enabling iterative refinement of control strategies through in-context learning, as also proposed by Mirchandani et al. (2023)²⁰. However, in contrast to previous approaches, our method does not depend on a human demonstrator. Instead, it builds its knowledge entirely through autonomous experience, continuously improving its performance using data generated in previous iterations.

Recent work in LLM-based agentic control has also explored hybrid architectures that combine language-based reasoning with structured tools such as simulators, digital twins, or robotics^{22–25}. In these approaches, LLMs typically function as planners or high-level policy generators, proposing sequences of tool invocations or the composition of predefined action primitives. While such methods are well suited for high-level reasoning, task decomposition, and domains where atomic actions or motor primitives are readily available, they are not designed to support low-level sensory–motor control.

In contrast, our approach focuses on generating entire control programs offline, enabling efficient high-frequency execution without further LLM involvement during rollout. This design choice trades off dynamic adaptability for runtime efficiency but is mitigated through an iterative refinement loop that allows the model to update its policy based on execution feedback across episodes. Compared to purely zero-shot methods or direct observation-to-action mapping, our methodology offers a scalable and computationally efficient framework for control policy synthesis, particularly in settings where low latency and structured adaptation over time are important.

The results obtained from classic control tasks in the Gymnasium library and the inverted pendulum problem in the MuJoCo library demonstrate that the proposed method consistently produces optimal or high-performing solutions. The analysis of the results underscores the value of combining the reasoning capabilities of large language models (LLMs) with a trial-and-error learning process driven by sensory-motor data and reward signals derived from the evaluation of candidate solutions. The approach proves effective with relatively compact models such as GPT-oss:120b and Qwen2.5:72b.

3. Method

To evaluate the capability of LLMs to directly control the actuators of embodied agents, we developed a structured prompting technique. We assessed its effectiveness using classic control tasks from the Gymnasium library (formerly Gym²⁶). The prompting methodology is described in Section 3.1, the control tasks considered are detailed in Section 3.2, the experimental conditions are outlined in Section 3.3, and the evaluation metrics are presented in Section 3.4. The code for replicating the experiments is publicly available together with all the supplementary material at <https://github.com/jtyska/llm-robotics-article/>.

3.1 Prompting technique

The proposed methodology comprises two primary phases. In the first phase, the model is prompted to generate a control policy capable of achieving a desired behavior under varying environmental conditions. This is based on a textual description that characterizes the agent’s body, sensors, actuators, environment, and task objectives. The description includes information on the agent’s morphology and explains the structure and semantics of the values encoded in the observation and action vectors—i.e., the sensory inputs and motor outputs.

In the second phase, the model is repeatedly prompted to refine its policy using the description of the performance outcomes of current and previous policies and sensory-motor data generated during policy evaluation.

To promote structured reasoning and ensure the synthesis of effective control policies, we adopted a step-by-step prompting strategy. First, the LLM is asked to reflect on the problem and propose a high-level control strategy (Prompt 1, Figure 1). Next,

it is instructed to translate this strategy into a set of IF-THEN-ELSE rules (Prompt 2). Finally, these rules are converted into executable Python code (Prompt 3). Preliminary experiments in which the LLM was directly instructed to generate Python code-without first articulating a high-level strategy and formulating it in a set of unambiguous rules-resulted in poor-quality output (data not shown), underscoring the importance of the structured prompting methodology.

Following the generation of the initial policy, the model undergoes an iterative improvement phase. It is prompted to revise its policy based on the same initial information, augmented with descriptions of the current, previous, and best control policies generated so far (when available), along with their associated performance scores and the sensory-motor data collected during the evaluation of the current policy (Prompt 4). This iterative refinement allows the model to adjust its control strategy and observe the impact of each modification. Due to token length constraints inherent in LLMs, the sensory-motor data included in each prompt are limited to the final 20 steps of a single evaluation episode.

An overview of this prompting methodology is illustrated in Figure 1, while the full prompt texts are provided in Supplementary Figure 1 in Appendix A.

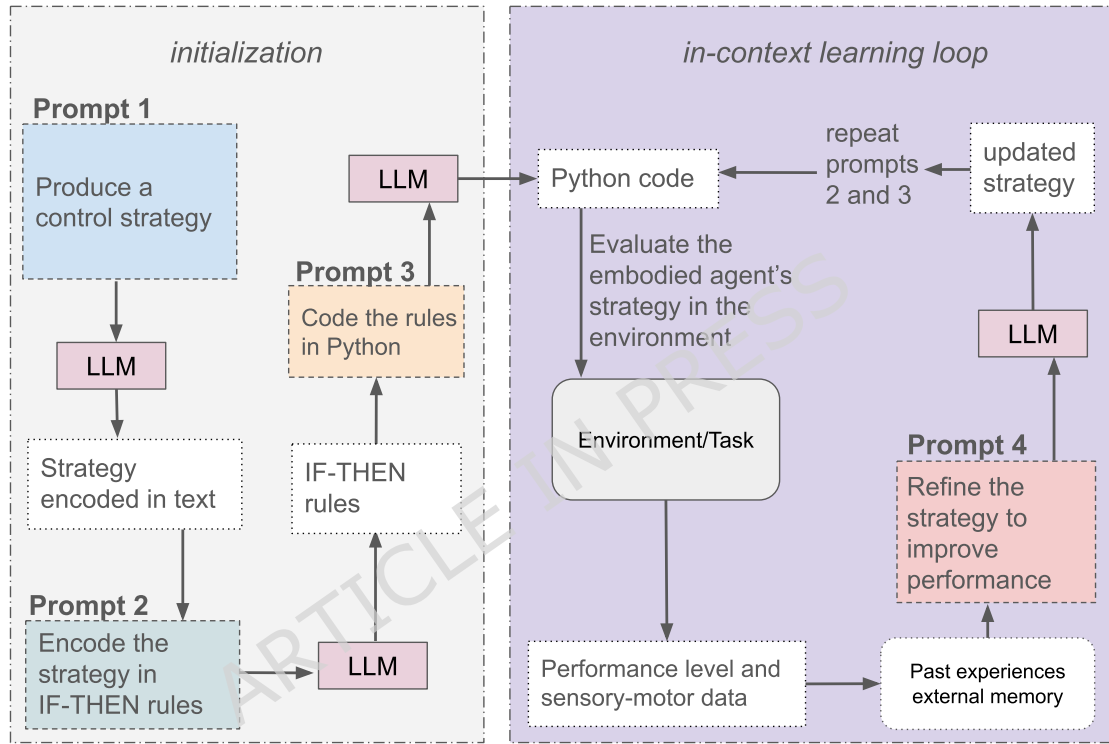


Figure 1. Overview of the prompting method enabling LLMs to directly control the actuators of embodied agents.

It is important to emphasize that the agent and task descriptions provided in the prompts do not include all the details necessary to generate a complete solution. For instance, in one of the tasks we used, the pole balancing, critical parameters such as pole length and torque magnitude were deliberately omitted. This omission illustrates a fundamental challenge in embodied control compared to mathematical or abstract reasoning tasks: the number of relevant variables is substantially larger, making it generally impractical to specify every detail of the control problem explicitly.

Nevertheless, our experiments demonstrate that such missing information can be inferred during the iterative learning process through the analysis of sensory-motor data and the observed outcomes of alternative control strategies. This capacity to integrate symbolic knowledge—conveyed through pre-trained data, textual prompts, and reasoning—with sub-symbolic knowledge—extracted from interaction with the environment—is central to the effectiveness of LLMs in embodied control tasks. It enables the derivation of effective solutions even in the presence of incomplete problem specifications.

Our proposed methodology facilitates problem-solving by drawing on three complementary sources of knowledge: (i) the general knowledge and reasoning abilities of LLMs, (ii) insights acquired through trial-and-error learning, and (iii) reasoning based on the observed effects of incremental policy modifications. This combination supports the generation of initial control strategies that already incorporate structural meaningful rules, thus providing a partial solution from the outset. Moreover, the model reasoning capabilities, in conjunction with performance feedback and sensory-motor data, allow for targeted and efficient policy refinements.

This approach offers a distinct advantage over traditional model-free learning methods, such as reinforcement learning, which typically start from random control strategies and rely on stochastic exploration to achieve improvement.

3.2 Control problems

We assessed the effectiveness of the proposed method using a suite of classic control tasks from Gymnasium (previously known as Gym²⁶). In addition to the five standard Gymnasium benchmark tasks—Acrobat, CartPole, MountainCar, MountainCarContinuous, and Pendulum—we also included the InvertedPendulum task from the MuJoCo library. We deliberately focused on these traditional, well-understood environments because our goal is to assess a novel capability: whether LLMs can directly generate functional controllers for embodied agents and whether Iterative Policy Refinement can reliably improve such controllers. These relatively simple benchmarks provide a clear and interpretable baseline for establishing viability before addressing more complex control scenarios. We emphasize that, although these tasks lie at the simpler end of standard continuous-control benchmarks, they nonetheless capture several fundamental challenges of sensory–motor control, including partial observability, delayed reward, nonlinear dynamics, and stabilization under perturbations. As such, they are well suited for validating the core methodological contribution of this work.

The choice of environments was also motivated by practical considerations related to computational cost. Evaluating an LLM-based policy generation framework is substantially more expensive than standard learning loops, as it requires repeated environment rollouts, controller execution, and multiple cycles of LLM prompting and refinement. Conducting the large-scale experimental sweep needed for a robust assessment of Iterative Policy Refinement therefore necessitated environments with short episode lengths and fast simulation times. While success on these classic control tasks establishes an essential baseline, it does not by itself characterize the limits of LLM-based control in higher-dimensional, contact-rich domains such as the Humanoid task. Extending the proposed approach to these more challenging environments is a natural next step, but doing so will require additional engineering and computational resources—such as longer-horizon evaluations, more expensive simulations, and potentially richer controller parameterizations—which we discuss in Section 5.

To ensure that the solutions generated by LLMs are not directly reproduced from memorized training data, we introduced two modified versions of the CartPole task: (i) Cartpole *1, which varies the encoding of the action space, and (ii) Cartpole *2, which alters the encoding of both the observation and action spaces. Agent morphologies (Supplementary Figure 2) and a detailed description of all of the tasks are provided in Appendix B. As we will demonstrate, the generation of original solutions is further supported by the analysis of the solution-generation process.

3.3 Experimental conditions

To evaluate the effectiveness of the proposed method, we conducted a comprehensive series of experimental runs, systematically varying the LLM, the temperature parameter—which controls the stochasticity in next-token prediction—and the control task. Each experimental condition was replicated ten times to ensure statistical robustness and allow for meaningful comparisons across models and settings.

The evaluated models included **LLaMa3.3** (70 billion parameters)²⁷, **Mistral-Large** (123 billion parameters)²⁸, **Qwen2.5** (72 billion parameters)²⁹, **DeepSeek-R1** (70 billion parameters)³⁰ and **GPT-oss:120b** (120 billion parameters)³¹. We also ran experiments with the GPT-oss:20b (20 billion parameters, which yielded markedly worse performance than the larger model). All models employed k-quantization techniques for weights quantization³², specifically the Q4_K_M variant. The models were hosted locally using an Ollama server¹ and executed on an NVIDIA DGX cluster with eight H100 GPUs.

Several methodological and practical considerations guided the selection of these specific models. First, all chosen models are open-source, ensuring complete transparency of the architectures and inference procedures and enabling local deployment without reliance on commercial APIs. This was a critical requirement for our experimental framework, which involved repeated, fine-grained evaluations under varying control conditions and the collection of sensory–motor interaction data. The open-source nature of these models also facilitated consistent experimental control and reproducibility across replications.

Second, we deliberately selected relatively compact models—ranging from 70 to 123 billion parameters—rather than the largest available LLMs. This choice reflects a trade-off between representational power and computational cost. While larger models may offer enhanced reasoning or generalization abilities, their high resource demands would have severely limited the number of experimental replications and parameter combinations that could be feasibly tested. By employing smaller yet high-performing models, we were able to perform multiple replications (ten per condition) and systematically investigate the impact of various temperature settings and task characteristics on model performance. This design choice enabled a more rigorous statistical assessment of the models’ behavior and of the stability of the learning process.

Moreover, these models represent state-of-the-art architectures within their respective families, combining efficient transformer backbones with advanced training methodologies such as mixture-of-experts routing, instruction tuning, and extended

¹<https://ollama.com/>

context handling. As such, they provide a representative spectrum of contemporary open-weight LLM designs, allowing us to examine whether the observed control capabilities are general across architectures or depend on specific model properties.

For the Cartpole*1 and Cartpole*2 tasks, we varied the model temperature across the set of [0.0, 0.4, 0.8, 1.6 and 3.2], thereby assessing the influence of stochasticity on the exploration–exploitation balance during policy generation. For all other control tasks, we limited models and temperature to the two setups that yielded the best results in preliminary experiments on the CartPole variants. The iterative learning process was run for 100 epochs for the CartPole tasks and 500 epochs for the InvertedPendulum task. The performance of the resulting control policies was evaluated by averaging the rewards obtained over 20 evaluation episodes, each initialized with randomly varied initial conditions. For the remaining tasks, training was limited to 50 epochs, and performance was assessed over 10 episodes.

This experimental design thus ensured a balanced combination of computational tractability, statistical rigor, and architectural diversity, providing a robust empirical foundation for evaluating the proposed LLM-based control methodology.

3.4 Evaluation metrics

Model performance in embodied control tasks spans multiple complementary dimensions that cannot be adequately captured by a single metric. Specifically, it is necessary to consider the proportion of experimental runs that successfully reach an optimal solution (success rate), the stability of the obtained controllers when evaluated across different episodes and random initializations (robustness), and the efficiency with which solutions are discovered during the iterative learning process (learning time). Because performance along these dimensions can vary independently, a model may converge rapidly but produce unstable controllers, or consistently identify solutions that lack robustness—no single metric alone can adequately identify the "best" model. We therefore designed a composite Figure of Merit (FoM) that integrates these complementary measures, enabling fair model-level comparisons and clear rankings across experimental conditions. This aggregate score serves solely as a concise summary for comparative purposes. Throughout our analysis, we also report the underlying metrics separately, allowing readers to examine each performance dimension in isolation and draw their own conclusions about model behavior.

To post-evaluate the results of the experiments, we designed the following evaluation metrics:

Average Reward: Quantifies the mean reward achieved by each tested policy throughout the iterated learning process across episodes and replications.

Success: Denotes the proportion of experimental replications in which the model successfully identified an optimal solution.

Learning Time: Evaluates the time taken for finding the optimal solution during the iterated learning process.

Robustness: Assesses the capability of the best policy to handle varying environmental conditions.

Figure of Merit (FoM): A composite index that provides an overall performance evaluation by aggregating Success, Robustness, and Learning Time.

The *Average Reward* of a model is computed by averaging the rewards obtained across all learning episodes and replications, as defined by the following equation:

$$\bar{R} = \frac{1}{N_{\text{reps}} \cdot N_{\text{episodes}}} \sum_{j=1}^{N_{\text{reps}}} \sum_{k=1}^{N_{\text{episodes}}} R^{(j,k)} \quad (1)$$

where:

- $N_{\text{reps}} = 10$: Number of replications,
- $N_{\text{episodes}} = 100$: Number of learning episodes,
- $R^{(j,k)}$: Reward for replication j and learning episode k .

The *Success* of a model is determined using the following expression:

Success (Q): Measures success frequency across replications:

$$Q_{\text{success}} = \frac{1}{N_{\text{reps}}} \sum_{j=1}^{N_{\text{reps}}} \mathbb{I} \left(\exists t \leq T_{\text{max}} : R^{(j,t)} = R_{\text{max}} \right) \quad (2)$$

where $N_{\text{reps}} = 10$ independent replications, $T_{\text{max}} = 100$ episodes, and $R_{\text{max}} = 500$ is the maximum reward. Range: [0, 1].

The *Learning Time* and *Robustness* metrics are calculated only for those replications in which the model successfully identified an optimal solution. These are defined by the following equations:

1. **Robustness** ($\bar{Q}_{\text{robustness}}$): For the first strategy that achieved maximum reward in each of the N_{reps} replications, measures strategy robustness across initial conditions:

$$\bar{Q}_{\text{robustness}} = \frac{1}{|\mathcal{S}_m|} \sum_{j \in \mathcal{S}_m} \frac{1}{N_{\text{episodes}}} \sum_{k=1}^{N_{\text{trials}}} \mathbb{I}(R^{(j,k)} = R_{\text{max}}) \quad (3)$$

where $N_{\text{episodes}} = 2000$, $R^{(j,k)}$ is the reward for the learning episode k in replication j , and $R_{\text{max}} = 500$ is the maximum reward. Range: $[0, 1]$.

2. **Learning Time** ($\bar{Q}_{\text{time}}$): For the first strategy that achieved maximum reward in each of the N_{reps} replications, measures convergence rapidity as the fraction of episodes needed to achieve the maximum reward:

$$\bar{Q}_{\text{time}} = \frac{1}{|\mathcal{S}_m|} \sum_{j \in \mathcal{S}_m} \frac{\min\{t \in [1, T_{\text{max}}] : R^{(j,t)} = R_{\text{max}}\}}{T_{\text{max}}} \quad (4)$$

where $T_{\text{max}} = 100$ is the maximum number of episodes, and $R_{\text{max}} = 500$. Range: $[0.01, 1]$.

For both *Robustness* and *Learning Time*, $\mathcal{S}_m$ denotes the subset of successful replications in which the model achieved the maximum reward at least once:

$$\mathcal{S}_m = \left\{ j \in \{1, \dots, N_{\text{reps}}\} \mid \exists t \leq T_{\text{max}} : R^{(j,t)} = R_{\text{max}} \right\}$$

The *Figure of Merit* for a model is computed using the following formulation:

$$\text{FoM} = \frac{\bar{Q}_{\text{stability}} \cdot (Q_{\text{success}})^2}{\bar{Q}_{\text{time}}} \quad (5)$$

To emphasize the importance of consistently discovering optimal solutions, the Success component is squared. This design choice penalizes models that fail to solve one or more replications, even if they exhibit high stability or fast convergence in other replications.

4. Results

Figure 2 and 3 present the results obtained for the two versions of the CartPole task under varying temperature settings. In Figure 2, performance is shown across increasing temperature values selected from the range $[0.0, 0.4, 0.8, 1.6, \text{ and } 3.2]$. Figure 3 displays performance at the optimal temperature level for each model. The GPT-oss:120b model achieves the highest performance, obtaining statistically superior results relative to all other models. In comparison, the Qwen2.5 and Llama3.3:70b models yield substantially lower performance, while Mistral-large123b and DeepSeek-r1:70b perform markedly worse and show no significant difference from one another.

Statistical significance was assessed using the Shapiro-Wilk test for normality, followed by a Kruskal-Wallis non-parametric ANOVA for overall group comparison. Post-hoc pairwise differences were evaluated using Dunn's test.

Figure 4 provides a detailed comparison of model performance across the two CartPole tasks at the optimal temperature setting. The GPT-oss:120b model significantly outperforms the other models with respect to the composite Figure of Merit evaluation metric (left panels). As shown in the right panels, this advantage is primarily attributable to its learning time, which is considerably faster than that of the other models. Moreover, this superiority stems from the success rate (the proportion of replications in which the model successfully discovers optimal solutions across both tasks). Qwen2.5:72b is the second-best performer with respect to the Figure of Merit. Its relative advantage over the other models arises from faster learning on the CartPole*1 task and a higher success rate on the CartPole*2 task.

The relatively poor performance of DeepSeek-R1, despite its strong capabilities in traditional reasoning tasks and in mathematical problems, suggests that embodied control and iterated learning rely on distinct forms of reasoning not captured by conventional benchmarks.

The control strategies proposed by the LLMs are the result of a dynamic reasoning process that combines domain-specific knowledge with iterative refinement. Consequently, they are not simply derived from solutions experienced in the training data. Evidence for this can be seen in the low initial performance of the Qwen2.5-72B model during the first iteration (see Table 1), where the maximum reward is 44.1 and 49.85 for the CartPole*1 and CartPole*2 tasks, respectively. Performance

improves significantly over subsequent iterations, reaching optimal rewards in 7 out of 10 replications in the CartPole*1 and in all 10 replications for CartPole*2. The three replications that did not reach optimal performance yielded maximum rewards of 66.85, 275.95, and 410.25, respectively (Table 1). The gpt-oss:120b model manages to generate optimal policies during the first iteration. However, this happens only in the case of the Cartpole*1 task and only in 4 out of 10 replications. These findings are further supported by the experimental design, which intentionally deviates from standard benchmark formulations to prevent the eventual usage of memorized solutions. The models thus engage in genuine problem-solving. As a point of reference, modern RL methods also achieve maximum performance on the CartPole benchmark^{33–35}.

As discussed, the optimal value of the temperature hyperparameter is both model- and task-dependent. However, to keep the computational cost feasible, we conducted additional experiments on the remaining tasks by testing the two best models in the CartPole tasks and the two best temperatures for each of them. The models tested were GPT-oss and Qwen2.5 and the best temperatures were, respectively, [0.0, 0.8], and [0.0, 1.6]. As in previous tests, each experimental condition was replicated 10 times.

Table 2 summarizes the results of these additional tasks. These problems do not have a predefined optimal reward. In any case, the GPT-oss model achieves by far the highest performance on four of the five tasks, while Qwen2.5 has the highest performance on the Inverted Pendulum, closely followed by GPT-oss. A notable case is the Pendulum task, where Qwen2.5's performance is notably poor compared to the high performance of GPT-oss. As with the CartPole tasks, initial performance in the first iteration is limited, underscoring the crucial role played by iterative learning and ensuring that the models do not simply retrieve predefined solutions available in the data used for the LLMs training process. As a point of reference, modern RL methods^{34–36} typically achieve average returns of approximately -263 for Pendulum, -138 for Acrobot, 950 for Inverted Pendulum, -110 and 90 on the discrete and continuous variants of MountainCar, respectively.

The solution process followed by the models can be illustrated through an analysis of the best-performing CartPole*2 replication carried out with Qwen2.5 (see also Appendix C). The complete experimental data, including logs, control rules, and videos, are available in the online repository: <https://jtyaska.github.io/llm-robotics-article/>.

The initial control strategy generated by the model, based solely on a textual description of the environment, agent, and task goal, consists of a structured set of rules. These rules attempt to correctly determine the agent's actions by considering the following states: (i) the direction in which the pole is leaning, (ii) the proximity of the cart to the left or right boundary, (iii) whether the pole's angular velocity exceeds certain thresholds, (iv) whether the cart's linear velocity exceeds certain thresholds, (v) whether the pole is (nearly) vertical.

Although the initial strategy includes a comprehensive set of plausible control rules, it fails to correctly prioritize the most critical factors and is structured in a way that only the rules based on the pole's angle are actually executed, while the remaining rules remain inactive. Consequently, the initial average reward is limited to 49.85 out of a maximum of 500.

Through iterative learning, the strategy is incrementally refined. In subsequent iterations, the average reward increases to 116.1 and 296.55, eventually reaching the maximum reward of 500 in the fourth iteration. These improvements arise by determining the action based on both the angular velocity of the pole and the angle of the pole, and by prioritizing the importance of the former. The model proactively analyzes the sensory-motor feedback from each iteration, identifies limitations, and proposes targeted modifications to enhance performance.

Despite these advancements, the models exhibit occasional errors and hallucinations, consistent with findings in prior work (e.g., Li et al., 2024³⁷; Wang et al., 2024⁶). For example, the model may misinterpret one sensor reading as another during introspective analysis. Such errors are observed in all replications but do not prevent the models from eventually discovering optimal control policies.

To assess the impact of the amount of sensory-motor data processed by the model during each iteration, we conducted a series of control experiments in which the number of sensory-motor steps was set to 5, 20, 50, and 100. The analysis was performed using the gpt-oss:20b model to limit the computational cost and on the Pendulum Task, i.e., the task LLMs for sensory-motor control that resulted in more difficulty. The results, presented in Figure 5, indicate that both too few and too many sensory-motor steps lead to reduced performance, with optimal performance achieved at an intermediate number of steps. A comparison of results from the gpt-oss:20b and gpt-oss:120b models using 20 sensory-motor steps, also presented in the figure, confirms that, as expected, larger models deliver superior performance.

To evaluate the effect of prompt accuracy, we performed a control experiment on the Pendulum task in which the task description was made more precise. In particular, we explicitly described how the sign of the action value determines the direction (clockwise or counter-clockwise) of the torque applied to the pendulum. As expected, the experiment using the more accurate prompt produced substantially better results. These findings are illustrated in the first two histograms of Figure 6. Providing a fully detailed description of the agent, its sensory-motor system, and the environment may be infeasible, especially in settings involving complex agents or environments. The model's ability to generate effective solutions even when provided with incomplete or partially inaccurate descriptions therefore represents an important strength of the proposed approach.

Finally, to assess the contribution of different key components of the method, we carried out a series of ablation experiments

in which one or more elements were removed from Prompt 4, which instructs the model to generate an improved control strategy. As shown in Figure 6, performance deteriorates drastically in the NoData condition, where all information from previous attempts and strategies is removed (i.e., the current, previous, and best strategies with their associated rewards, as well as the sensory-motor data collected during the evaluation of the current strategy). Removing only the sensory-motor data (NoSensoryMotorData), the previous strategy with its reward (NoPreviousStrategy), or the best strategy with its reward (NoBestStrategy) leads to a moderate reduction in performance. All conditions differ statistically from the baseline (MoreAccuratePrompt), except for the case where the previous strategy was not provided (NoPreviousStrategy). We assessed distributional normality using the Shapiro–Wilk test and subsequently applied the Mann–Whitney U test to evaluate statistical significance, as the data did not meet the assumption of normality.

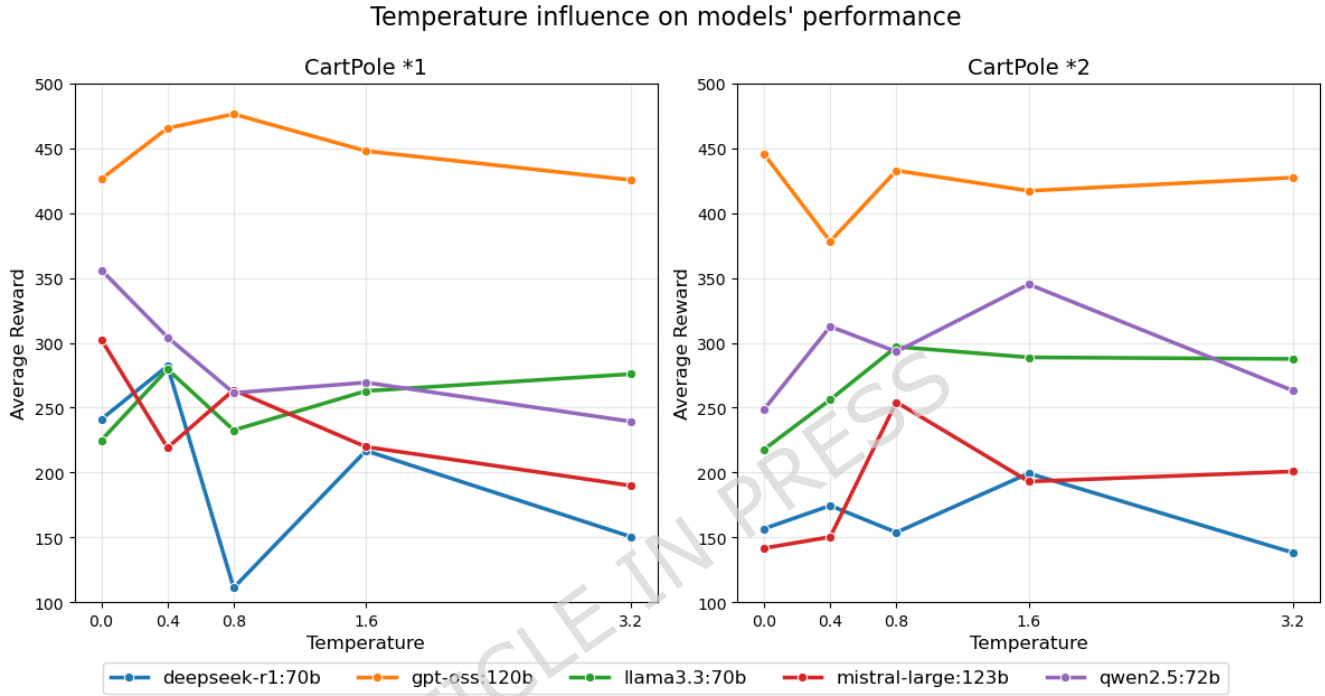


Figure 2. Average reward obtained for the CartPole1 (left) and CartPole2 (right) tasks. The horizontal axis represents the temperature value, while the vertical axis shows the average reward, computed over 10 replications. Line colors indicate the different models evaluated.

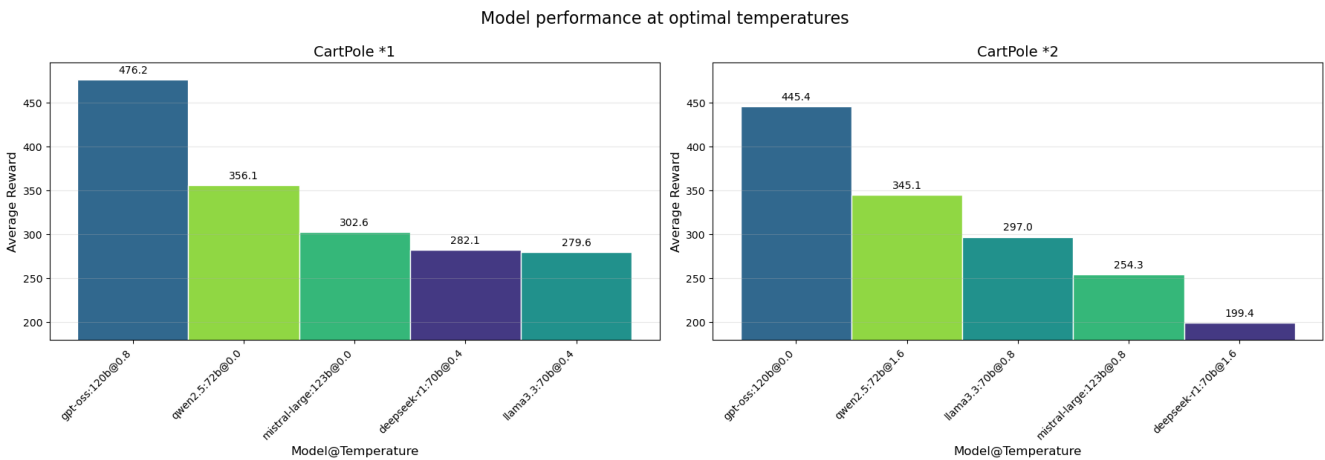


Figure 3. Average reward at the optimal temperature setting for the Cartpole*1 and Cartpole*2 tasks (left and right, respectively). The vertical axis represents the average reward, while the horizontal axis specifies the LLM used.

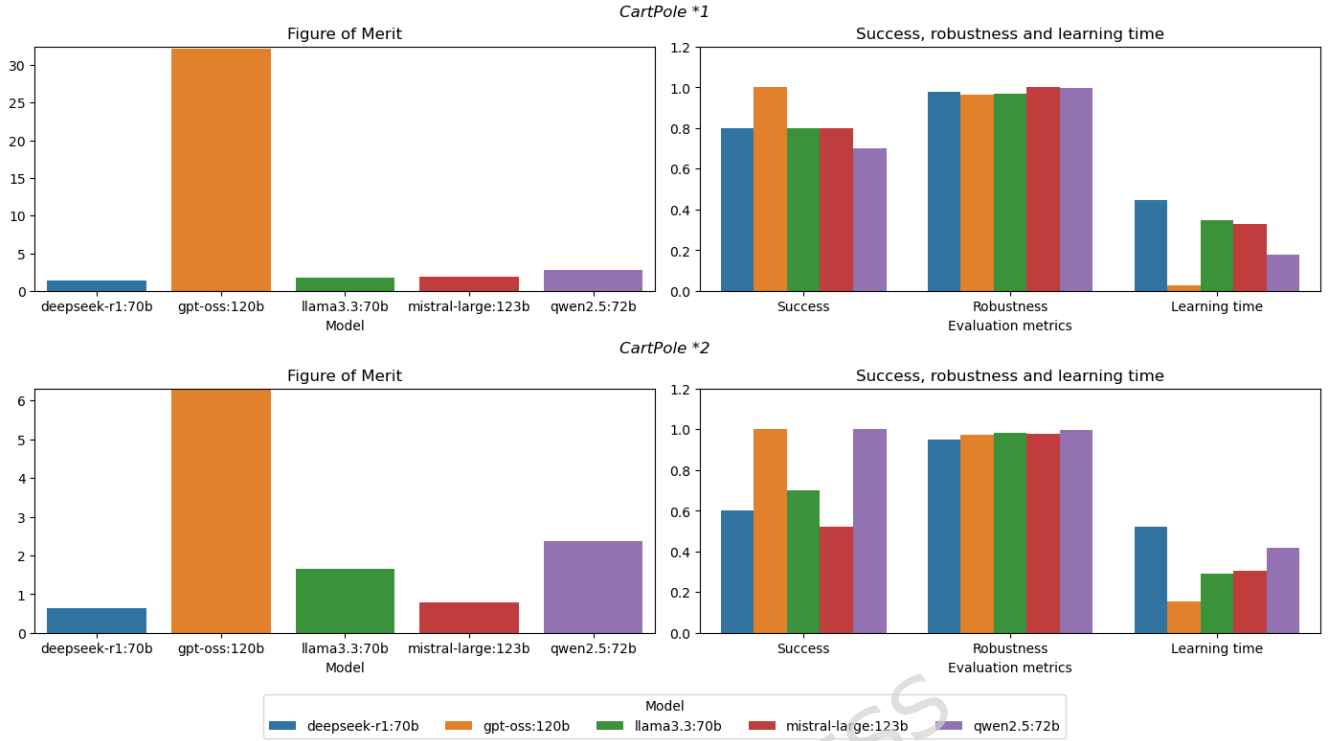


Figure 4. Performance metrics for the CartPole *1 (top) and CartPole *2 (bottom) tasks. The plots on the left display the Figure of Merit, while the plots on the right show Success, Robustness, and Learning Time. Histogram colors indicate the LLM used. Results are reported for the optimal temperature setting. Note that the y-axis scales differ between the left-hand plots.

5. Discussion

We presented a method that enables large language models (LLMs) to control embodied agents, allowing them to exhibit goal-directed behavior. Unlike approaches that rely on predefined motor primitives, our method generates control policies that directly map continuous observation vectors to continuous action vectors, thereby controlling the agent’s actuators directly. Moreover, the method operates without requiring model fine-tuning, thereby eliminating the need to collect large-scale sensory-motor datasets.

The proposed approach leverages the in-context learning capabilities of LLMs in conjunction with an iterative learning process. Initially, the model is prompted to generate a control strategy based on a verbal description of the agent, the environment, and the intended goal. This strategy is subsequently refined through repeated prompting, in which the model is asked to reflect on the performance of the current (and eventually previous) strategy using sensory-motor data obtained during evaluation, and to revise the strategy accordingly. This iterative refinement mechanism, supported by performance feedback and sensory-motor data, enables the model to identify and eliminate maladaptive control rules that may emerge from reasoning errors, hallucinations, or the inherent incompleteness of the problem description. We validated the proposed method using five relatively compact LLMs on a suite of classic control tasks from the Gymnasium library, as well as the inverted pendulum task from the MuJoCo library.

The results obtained in this study highlight the distinctive potential of large language models (LLMs) as controllers for embodied and situated agents. Unlike conventional approaches that rely on extensive training data or explicit demonstrations, LLMs can synthesize plausible initial control strategies on the fly based solely on a verbal description of the agent, its environment, and the task to be achieved. This ability stems from their vast knowledge and abstract reasoning skills, which allow them to infer the structure of plausible, effective control rules. Moreover, LLMs can serve as reasoning engines capable of hypothesizing, testing, and refining behavioral strategies through iterative interaction.

A central advantage of this paradigm is that it enables the rapid initialization of goal-directed behavior. When prompted with a textual specification, LLMs can draw on prior conceptual and procedural knowledge to propose a structured control policy that already encodes meaningful relationships between sensory and motor states. This contrasts sharply with reinforcement learning (RL) or other model-free methods, which typically begin from random policies and require extensive exploration to

Table 1. Rewards obtained during the first iteration and during the iteration that achieved the highest reward. For both the first and best iterations, the table reports the average rewards across all replications, as well as the rewards achieved in the best and worst replications. The results were obtained on the CartPole*1 and CartPole*2 tasks using the deepseek-r1:70b, llama3.3:70b, mistral-large:123b, qwen2.5:72b, and GPT-oss:120b models with their optimal temperature settings. Reward values range from 0 to 500, with higher values indicating better performance.

Environment	Model	First iteration reward			Best iteration reward		
		Worst replication	Best replication	Average	Worst replication	Best replication	Average
CartPole *1	deepseek-r1:70b	8.70	45.75	12.60	473.75	500	495.78
	llama3.3:70b	9.20	172.65	74.45	352.35	500	484.46
	mistral-large:123b	8.60	8.85	8.78	52.90	500	429.50
	qwen2.5:72b	41.55	44.10	43.05	66.85	500	425.31
	gpt-oss:120b	42.15	500.00	344.32	500.00	500	500.00
CartPole *2	deepseek-r1:70b	9.60	40.15	18.57	376.65	500	472.46
	llama3.3:70b	8.80	315.05	71.24	470.30	500	495.80
	mistral-large:123b	9.00	190.80	41.54	124.95	500	415.19
	qwen2.5:72b	8.70	49.85	18.31	500.00	500	500.00
	gpt-oss:120b	9.8	486.1	245.99	500.00	500	500.00

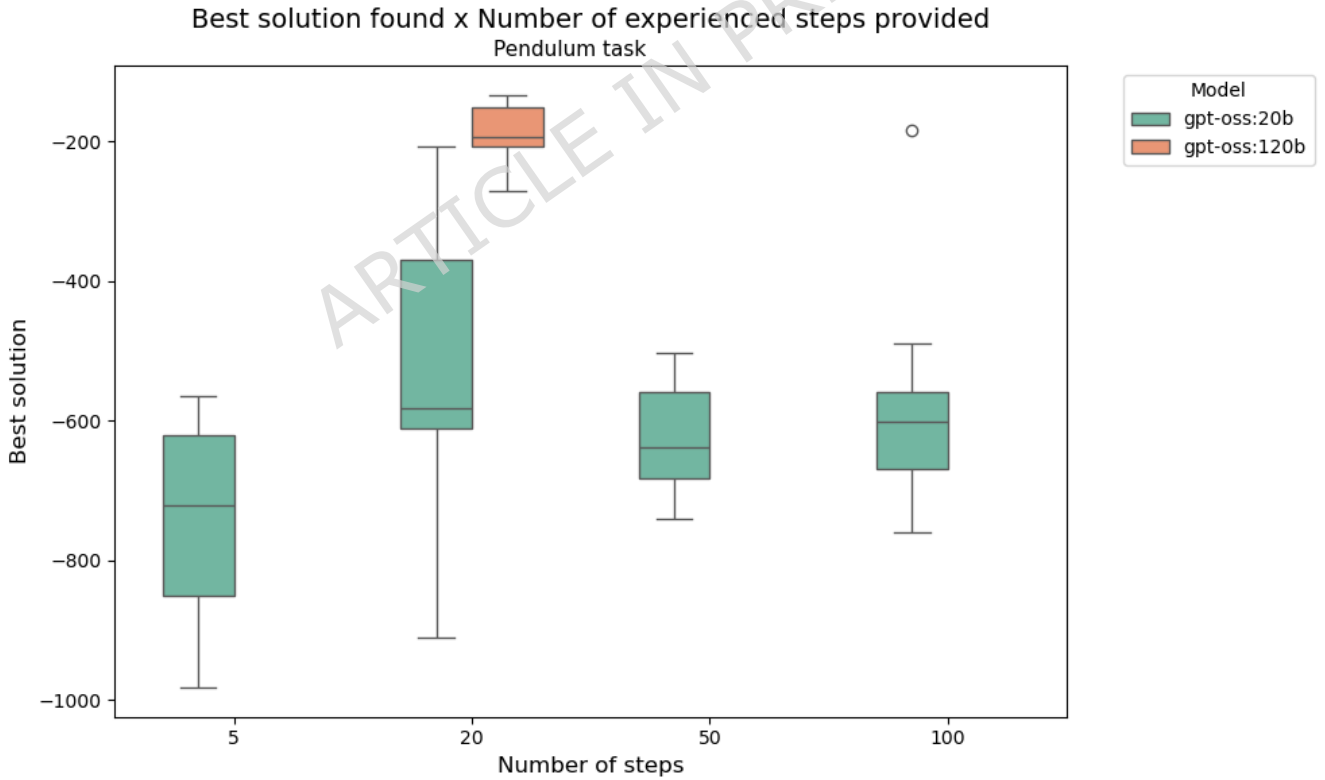


Figure 5. Experiments conducted by varying the amount of sensory-motor data provided to the model. All experiments were performed on the Pendulum task. The green and orange boxplots correspond to the gpt-oss:20b and gpt-oss:120b models, respectively. Each boxplot summarizes the results of 10 replications.

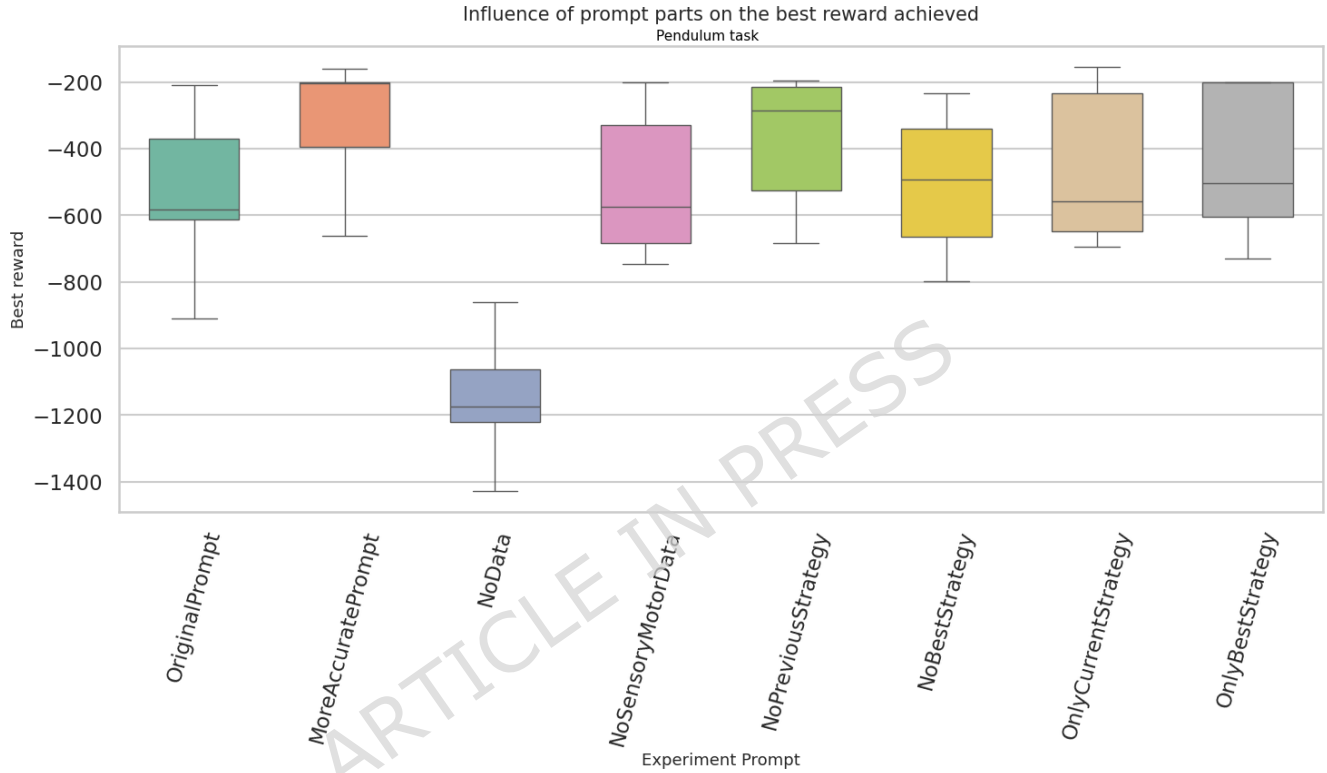


Figure 6. Experiments conducted by varying the accuracy of the task description in the prompt and by ablating model components. All data were collected on the Pendulum task using the gpt-oss:20b model. **OriginalPrompt:** baseline task description used in the main experiments; **MoreAccuratePrompt:** revised, more precise task description with ambiguities removed and additional task-relevant details; **NoData:** Prompt 4 is omitted entirely (no current and prior strategies, rewards, or sensory-motor data); **NoSensoryMotorData:** sensory-motor data are omitted; **NoPreviousStrategy:** the previous strategy with its associated reward is omitted; **NoBestStrategy:** the best strategy with its associated reward is omitted; **OnlyCurrentStrategy:** only the current strategy with its associated reward and sensory-motor data is included; **OnlyBestStrategyData:** only the best strategy with its associated reward is included.

Environment	Model	Optimal Temperature	Mean Learning Time	Mean Best Reward	Mean Initial Reward	Mean Improv. Ratio
Acrobot	gpt-oss	0.8	0.03	-77.30	-500.00	0.99
	qwen2.5	1.6	0.04	-108.24	-500.00	0.92
Inverted Pendulum	gpt-oss	0.0	0.31	738.58	12.45	0.73
	qwen2.5	1.6	0.19	829.66	5.05	0.83
MountainCar Continuous	gpt-oss	0.8	0.04	94.81	-99.90	0.99
	qwen2.5	0.0	0.04	62.22	-56.26	0.78
MountainCar Discrete	gpt-oss	0.8	0.16	-111.54	-200.00	0.87
	qwen2.5	1.6	0.07	-166.24	-200.00	0.33
Pendulum	gpt-oss	0.8	0.18	-190.31	-1725.44	0.96
	qwen2.5	0.0	0.13	-911.81	-1334.76	0.35

Table 2. Performance results for the Acrobot, Inverted Pendulum, MountainCar Continuous, MountainCar Discrete, and Pendulum tasks obtained using GPT-oss and Qwen2.5 models with optimal temperature settings. For each task, the table reports the optimal temperature, mean learning time, mean best reward, mean initial reward, and mean improvement ratio. “Mean” indicates that these four values are averaged across replications. The improvement ratio is the percentage gain over the initial performance, measured against the task’s maximum reward (when defined) or otherwise the best reward achieved for that task across all experiments. In all tasks, performance improves with higher values.

converge on a viable solution. By starting from an informed, reasoned hypothesis, LLM-based agents can bypass the most computationally expensive early phases of exploration, thus accelerating the learning process and reducing the amount of interaction data needed to reach optimal or near-optimal performance.

Moreover, the reasoning abilities of LLMs make it possible to analyze and interpret the outcomes of previous control attempts in a qualitatively different manner from statistical learners. During iterative refinement, the model can explicitly describe and justify modifications to its own strategy—inferring, for instance, which variables have the greatest influence on success or which control rules should be prioritized or suppressed. This metacognitive capability allows LLMs to apply targeted changes that improve behavioral efficacy without requiring large numbers of stochastic trials. The ability to combine explicit reasoning about system dynamics with empirical feedback from sensory-motor data provides a powerful form of hybrid learning that integrates symbolic and sub-symbolic knowledge sources.

From a broader perspective, this integration suggests that LLM-based control may substantially reduce the dependency on manually curated datasets and physical-world demonstrations that currently constrain data-driven robotics. Instead, control strategies can be generated, evaluated, and iteratively refined, guided by the LLM’s internal knowledge of physical regularities, task structures, and causal dependencies. As demonstrated in our experiments, even relatively compact models such as GPT-oss:120b and Qwen2.5:72b were able to infer missing parameters and converge toward optimal control solutions. This indicates that the knowledge encoded in modern language models can serve as a versatile prior for embodied learning, bridging the gap between high-level reasoning and continuous motor control.

Despite these promising results, several limitations remain that constrain the scalability and practical deployment of the proposed approach. First, the computational cost associated with employing very large models—often comprising tens or hundreds of billions of parameters—represents a significant bottleneck. Although our experiments show that relatively compact models such as GPT-oss:120b and Qwen2.5:72b already achieve competitive performance, real-world applications requiring continuous and high-frequency control would demand even greater computational efficiency. Future research should therefore investigate strategies for model distillation, parameter-efficient fine-tuning, or modular architectures that allow smaller specialized models to handle distinct aspects of embodied control while retaining access to shared linguistic and reasoning capabilities.

A second limitation concerns the intrinsic restriction imposed by the finite context window of current LLMs. The number of input tokens that can be processed at once limits both the amount of sensory-motor data collected during policy evaluation and the quantity of performance feedback that can be incorporated during iterative learning. In our experiments, including only 20 sensory-motor steps yielded the best results. However, more complex problems and larger LLMs may require substantially larger amounts of data. Expanding the effective memory span of the models or developing hierarchical compression mechanisms for

sensory-motor sequences will be essential for extending the applicability of LLM-based control to more complex, long-horizon tasks.

Building on these considerations, a deeper investigation of scalability is crucial. As task complexity increases, particularly in the form of higher-dimensional state and action spaces, longer planning horizons, or partially observable environments, the computational and contextual costs related to the use of LLM grow substantially. Larger observation and action spaces require more extensive sensory-motor representations, thereby expanding the size of the input prompt and increasing the token budget needed to encode both the policy and its historical context. Similarly, long-horizon tasks necessitate broader windows of past experience to capture temporal dependencies, goal structure, or delayed effects. While including more interaction history can improve planning, it also raises the risk of overwhelming the model with low-level numerical data, which may distract from the high-level task objective and degrade reasoning quality.

A key mechanism for addressing these challenges is the Iterative Policy Refinement framework itself. By actively exploring and reflecting over successive policy evaluations, the LLM agent can gradually discover and incorporate missing information, allowing it to cope with initially incomplete state representations or partial observability. This iterative design effectively enables the agent to “fill in the gaps” of its world model through structured trial-and-error. To further support scaling to more complex control problems, recent developments in agentic AI, such as the Model Context Protocol, offer promising architectural solutions. These protocols provide standardized methods for dynamically managing large and structured context, enabling the integration of modular memory, subtask decomposition, and feedback-guided control. Incorporating such capabilities could systematically mitigate the scalability limits identified here and support LLM-based agents operating in high-dimensional, sequential, and partially observable domains.

A third limitation involves the current dependency on manually constructed prompts. Although structured prompting proved crucial for eliciting coherent control strategies and guiding iterative refinement, it still requires significant human expertise to design effective prompt templates and ensure consistent interpretation by the model. This manual component can hinder scalability and introduce subjective biases into the learning process.

Looking forward, future work should focus on automating and optimizing the prompt design process. One promising direction is the development of adaptive prompting methods capable of dynamically selecting the most relevant information to include in each iteration based on task performance, uncertainty estimates, or changes in environmental conditions. Automated prompt engineering frameworks, such as AutoPrompt³⁸ and related meta-learning approaches, could be leveraged to discover optimal prompting strategies without human intervention. By coupling such methods with mechanisms for intelligent data selection—prioritizing the most informative sensory-motor samples and performance summaries—the overall efficiency and autonomy of LLM-driven control systems could be further enhanced. The framework would also benefit from the use of more advanced language models and could be extended to dual-agent architectures, in which one model generates control strategies while a second model critiques them, identifies weaknesses, and provides constructive feedback^{39,40}.

Another promising research direction involves identifying the knowledge and skills that are specifically relevant to embodied and situated control, as well as assessing the extent to which existing LLMs possess these capabilities. Plausible candidates include the ability to handle incomplete problem descriptions and to model the physical world.

Ultimately, overcoming these limitations will be key to transforming LLM-based control from a proof of concept into a scalable and generalizable paradigm for embodied intelligence.

References

1. Huang, W. *et al.* Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608* (2022).
2. Brown, T. *et al.* Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, vol. 33, 1877–1901 (2020).
3. Chen, M. *et al.* Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).
4. Driess, D. *et al.* Palm-e: An embodied multimodal language model. In *International Conference on Machine Learning*, 8469–8488 (PMLR, 2023).
5. Zeng, Z. & Mao, W. A comprehensive empirical study of vision-language pre-trained model for supervised cross-modal retrieval. *arXiv preprint arXiv:2201.02772* (2022).
6. Wang, K. *et al.* On the planning abilities of openai’s o1 models: Feasibility, optimality, and generalizability. In *Language Gamification-NeurIPS 2024 Workshop*.
7. West-Eberhard, M. J. *Developmental plasticity and evolution* (Oxford University Press, 2003).
8. Nolfi, S. *Behavioral and cognitive robotics: An adaptive perspective* (Stefano Nolfi, 2021).

9. Sutton, R. S. & Barto, A. G. *Reinforcement Learning: An Introduction* (The MIT Press, Cambridge, MA, 2018), 2 edn.
10. Kober, J., Bagnell, J. A. & Peters, J. Reinforcement learning in robotics: A survey. *The Int. J. Robotics Res.* **32**, 1238–1274, DOI: [10.1177/0278364913495721](https://doi.org/10.1177/0278364913495721) (2013).
11. Lillicrap, T. P. *et al.* Continuous control with deep reinforcement learning. *arXiv preprint arXiv:1509.02971* (2015).
12. Haarnoja, T., Zhou, A., Abbeel, P. & Levine, S. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In *Proceedings of the 35th International Conference on Machine Learning*, vol. 80 of *Proceedings of Machine Learning Research*, 1861–1870 (PMLR, 2018).
13. Kalashnikov, D. *et al.* Qt-opt: Scalable deep reinforcement learning for vision-based robotic manipulation. *arXiv preprint arXiv:1806.10293* (2018).
14. Levine, S., Kumar, A., Tucker, G. & Fu, J. Offline reinforcement learning: Tutorial, review, and perspectives on open problems. *arXiv preprint arXiv:2005.01643* (2020).
15. Fu, J., Kumar, A., Nachum, O., Tucker, G. & Levine, S. D4rl: Datasets for deep data-driven reinforcement learning. *arXiv preprint arXiv:2004.07219* (2020).
16. Kumar, A., Zhou, A., Tucker, G. & Levine, S. Conservative q-learning for offline reinforcement learning. In *Advances in Neural Information Processing Systems 33 (NeurIPS)* (2020). ArXiv:2006.04779.
17. Brohan, A. *et al.* Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817* (2022).
18. Chen, X. *et al.* Pali-x: On scaling up a multilingual vision and language model. *arXiv preprint arXiv:2305.18565* (2023).
19. Bubeck, S. *et al.* Sparks of artificial general intelligence: Early experiments with gpt-4 (2023).
20. Mirchandani, S. *et al.* Large language models as general pattern machines. *arXiv preprint arXiv:2307.04721* (2023).
21. Haddadin, S. The franka emika robot: A standard platform in robotics research. *IEEE Robotics & Autom. Mag.* (2024).
22. Kannan, S. S., Venkatesh, V. L. N. & Min, B.-C. Smart-llm: Smart multi-agent robot task planning using large language models. In *2024 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 12140–12147, DOI: [10.1109/IROS58592.2024.10802322](https://doi.org/10.1109/IROS58592.2024.10802322) (2024).
23. Gill, M. S., Vyas, J., Markaj, A., Gehlhoff, F. & Mercangöz, M. Leveraging llm agents and digital twins for fault handling in process plants. In *2025 IEEE 30th International Conference on Emerging Technologies and Factory Automation (ETFA)*, 1–8, DOI: [10.1109/ETFA65518.2025.11205597](https://doi.org/10.1109/ETFA65518.2025.11205597) (2025).
24. Vyas, J. & Mercangöz, M. Autonomous industrial control using an agentic framework with large language models. *IFAC-PapersOnLine* **59**, 349–354 (2025).
25. Ao, J., Wu, F., Wu, Y., Swiki, A. & Haddadin, S. Llm-as-bt-planner: Leveraging llms for behavior tree generation in robot task planning. In *2025 IEEE International Conference on Robotics and Automation (ICRA)*, 1233–1239 (IEEE, 2025).
26. Towers, M. *et al.* Gymnasium: A standard interface for reinforcement learning environments. *arXiv preprint arXiv:2407.17032* (2024).
27. Grattafiori, A. *et al.* The llama 3 herd of models. *arXiv preprint arXiv:2407.21783* (2024).
28. Mistral AI Team. Mistral large instruct 2407. Hugging Face model card (2025). Accessed 2026-01-29.
29. Yang, A. *et al.* Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115* (2024).
30. Guo, D. *et al.* Deepseek-r1: Incentivizing reasoning capability in llms via reinforcement learning. *arXiv preprint arXiv:2501.12948* (2025).
31. Agarwal, S. *et al.* gpt-oss-120b & gpt-oss-20b model card. *arXiv preprint arXiv:2508.10925* (2025).
32. Kim, S. *et al.* Squeezellm: Dense-and-sparse quantization. *arXiv preprint arXiv:2306.07629* (2023).
33. Shankar, K., Louw, W. & Cohen, K. On-policy optimization of anfis policies using proximal policy optimization. *arXiv preprint arXiv:2507.01039* (2025).
34. Asri, Z. E., Sigaud, O. & Thome, N. Physics-informed model and hybrid planning for efficient dyna-style reinforcement learning. *arXiv preprint arXiv:2407.02217* (2024).
35. OpenAI. Gym leaderboard. GitHub Wiki (2024). Last edited Apr 30, 2024. Accessed 2026-01-29.
36. Chou, P.-W., Maturana, D. & Scherer, S. Improving stochastic policy gradients in continuous control with deep reinforcement learning using the beta distribution. In *International conference on machine learning*, 834–843 (PMLR, 2017).

37. Li, M. *et al.* Embodied agent interface: Benchmarking llms for embodied decision making. *Adv. Neural Inf. Process. Syst.* **37**, 100428–100534 (2024).
38. Shin, T., Razeghi, Y., Logan IV, R. L., Wallace, E. & Singh, S. Autoprompt: Eliciting knowledge from language models with automatically generated prompts. *arXiv preprint arXiv:2010.15980* (2020).
39. Diallo, A., Bikakis, A., Dickens, L., Hunter, A. & Miller, R. Rule-guided feedback: Enhancing reasoning by enforcing rule adherence in large language models. *arXiv preprint arXiv:2503.11336* (2025).
40. Renze, M. & Guven, E. Self-reflection in llm agents: Effects on problem-solving performance. *arXiv preprint arXiv:2405.06682* (2024).

Acknowledgements

This work was carried out using computational resources provided by FAPESC – Foundation for Research and Innovation Support of the State of Santa Catarina, under grant number 2024TR000090. JTC acknowledges support from the Brazilian National Research Council (CNPq) through grant No. 200013/2025-4. JTC and SN acknowledge support from the European Research Program “Future Artificial Intelligence Research – FAIR” (CUP B53C22003630006, Project Code MIUR PE00000013).

Author contributions statement

JTC AND SN conceived the experiment(s), JTC conducted the experiment(s), and JTC and SN analyzed the results. Both authors prepared and reviewed the manuscript.

Data Availability

Code for replicating the experiments and supplementary material are available at <https://github.com/jtyska/llm-robotics-article/>

Competing Interests

The authors declare no competing interests.

ARTICLE IN PRESS